ChatGPT

Efficient Serving of Large Language Models: KV Cache, Speculative Decoding, and Quantization

Abstract: Modern large language models (LLMs) offer remarkable capabilities but impose heavy inference costs in latency and memory. Three orthogonal classes of techniques have emerged to accelerate LLM serving: key-value (KV) cache optimization, speculative decoding, and model quantization. KV caching stores past token representations to avoid redundant computation, while speculative decoding uses a lightweight "draft" model to propose future tokens in parallel with verification by the full model. Quantization reduces precision (e.g. to 8- or 4-bit) in weights, activations, or even KV caches to save memory and increase throughput. This survey systematically reviews each family. We organize KV cache methods into token-selection (cache eviction/compression), hybrid memory (offload, multi-tier), and cross-context reuse strategies; we examine speculative decoding variants including single-draft, parallel multi-draft, and self-speculative algorithms; and we survey quantization methods for weights, activations, and caches. Throughout, we report empirical results from recent literature, quantify trade-offs (accuracy vs speed), and discuss challenges (accuracy drop, acceptance rates, hardware support). We identify open problems such as adaptive multi-stage pipelines and integrated hardware/software support. Our goal is to provide a comprehensive technical reference (with ~200 references) on LLM-serving optimizations for practitioners and researchers.

Introduction

Large language models (LLMs) based on the Transformer architecture have revolutionized NLP, but their autoregressive inference is costly: generating K new tokens requires K serial passes through the model. To mitigate this, modern implementations use a **KV cache**, storing each token's key and value vectors so they need not be recomputed at every step【63†L93-L101】. This reduces the per-token compute from quadratic to linear in context length, accelerating generation for long sequences【63†L93-L101】. However, KV caches grow with context length and model size: for example, a 7B-parameter model with 128K context tokens requires on the order of 64 GB of cache, exceeding typical GPU memory【31†L184-L190】. Thus, KV cache usage presents a memory bottleneck that has spurred many optimization efforts.

Independently, **speculative decoding** techniques have appeared to accelerate generation. In speculative decoding, a smaller "draft" model rapidly proposes one or more future tokens, and the large target model verifies them jointly【57†L29-L37】. This allows parallelizing the sequential sampling process. For example, Leviathan et al. show that speculative decoding can achieve a **2–3× speedup** on T5-XXL without loss of output distribution【57†L29-L37】【57†L38-L40】. Variants such as self-speculation and multi-draft strategies have been developed to further push parallelism or improve efficiency【61†L28-L36】【41†L29-L37】. Crucially, speculative methods trade off extra work (running the draft model and verifying) for reduced wall-time by overlapping execution.

A third orthogonal approach is **quantization**: reducing numeric precision of the model to speed up computation and shrink memory. 8-bit quantization of matrix multiplies (e.g. LLM.int8【54†L29-L37】) or 4-bit weights have become practical for LLM inference. Recent advances (GPTQ, AWQ, etc.) achieve nearly lossless 4-bit weight quantization with minimal accuracy drop【42†L19-L24】【17†L121-L129】. Quantization not only applies to model weights and activations but can also compress KV caches (e.g. 2-bit KV quantization in KIVI【44†L41-L49】). These methods exchange small accuracy loss for 2–4× speedups or memory reductions.

This survey paper provides a deep technical review of these three families. In the **Methods** section, we organize and detail the state-of-the-art techniques for KV caching (Section 2.1), speculative decoding (2.2), and quantization (2.3). Each technique is linked to specific algorithmic strategies and empirical results. In Section 3 we discuss combined approaches, practical trade-offs, and open challenges (e.g. achieving high acceptance rates in speculation, choosing quantization schemes under hardware limits). Section 4 concludes with future directions. All statements are grounded in recent academic work 【57†L29-L37】【30†L78-L86】【17†L121-L129】.

Methods

2.1 Key-Value Cache Optimization

2.1.1 Token-Level Cache Pruning and Compression

Because the KV cache's memory scales with context length linearly, many approaches **evict or compress** unimportant key-value entries. A general intuition is that most LLM attention focuses on a few "heavy-hitter" tokens, so pruning low-impact entries saves memory with little accuracy loss. For example, H_2O (Heavy-Hitter Oracle) dynamically tracks accumulated attention scores. It greedily retains "heavy-hitter" tokens (high total attention) and recent tokens, discarding low-scoring ones【37†L41-L47】【34†L303-L310】. Experiments show this balances recency and importance: retaining only 20% heavy-hitters on OPT models yields up to **1.9× latency reduction** and large throughput gains【37†L41-L47】. Similarly, SnapKV uses a voting scheme: it slides an "observation window" over the prompt, computes attention-derived scores per token, then clusters and keeps the top features and their neighbors【34†L320-L329】【39†L52-L60】. In practice, SnapKV achieves **3.6× faster decoding** and **8.2× less memory** on 16K inputs (with minimal accuracy drop) by selecting ~50% of tokens as a compressed cache【39†L52-L60】. Other token-pruning methods include continual context distillation (InfiniPot) and attention-free hashing (HashEvict) that use novelty scores or LSH to drop predictable or least-similar tokens【34†L361-L370】【34†L379-L388】. A recent survey confirms that eviction/compression can trade memory for speed, but their effectiveness is highly task-dependent【31†L112-L119】【34†L303-L312】.

2.1.2 Hybrid Memory and Sparse Attention

Beyond selective eviction, some systems offload or hierarchically store KV caches. For instance, frameworks like FlexGen and PagedAttention spill parts of the cache to CPU memory or SSD when GPU RAM is scarce. Such hybrid designs overlap communication with computation to relieve GPU pressure. Moreover, new attention architectures (e.g. linearized or sparse attention) implicitly reduce KV usage by computing only partial contexts. In practice, frameworks like vLLM implement mixed techniques: they may shard keys across GPU and CPU and apply limited sparsity heuristics to drop negligible contexts 【30†L112-L119】【63†L77-L84】. Extensive comparisons (e.g. vLLM vs. InfiniGen vs. H_2O) show that the best strategy depends on context length, batch size, and hardware【63†L77-L84】【63†L119-L127】. No single method dominates all settings; indeed, Xu et al. note that "the optimal strategy depends on context length, hardware constraints, and workload," suggesting adaptive pipelines that combine eviction, compression, and offloading as needed【30†L83-L90】.

2.1.3 Cross-Context and Multi-Agent Reuse

A new dimension of KV cache optimization arises in **multi-agent or interactive** settings, where one agent's outputs become another's inputs. In such pipelines (e.g. chains of reasoning or dialogue), shared tokens often reappear across stages but under different prefixes. Simply prefix-caching fails if token positions shift. Recent methods explicitly align and reuse KV caches across contexts. For example,

KVCOMM (NeurIPS 2025) introduces a pool of “anchor” cache snippets capturing how keys deviate under different prefixes. At inference, KVCOMM matches new inputs to these anchors and reuses up to 70% of previous cache, yielding **~7.8× speedup** and dramatic TTFT reduction with negligible quality loss【59 † L51-L56】. Similarly, RelayCaching (arXiv 2026) observes that cache deviations are sparse across layers and tokens, so it selectively recomputes only the changed parts. This yields over **90% cache reuse** across agents and commensurate latency reductions【48 † L672-L675】. These schemes are **training-free** and orthogonal to the above methods; they address the redundant “prefill” computations that otherwise explode in multi-turn workflows【59 † L51-L56】.

2.1.4 Quantizing the KV Cache

Yet another twist is to apply quantization directly to the cache entries. Since keys and values are floating-point vectors, they can be coarsely quantized to save memory bandwidth. KIVI (ICML 2024) exemplifies this by using a novel 2-bit asymmetric quantization: it quantizes the key cache per-channel and value cache per-token, preserving accuracy while storing only 1/2.6 of the bytes【44 † L41-L49】. KIVI allows processing up to 4× larger batches and achieves **2.35–3.47× throughput** on real LLM workloads with negligible loss【44 † L41-L49】. More generally, quantizing the KV cache trades a bit of fidelity for memory relief, enabling schemes like **QuantSpec** (discussed below) to apply 4-bit caches in speculative decoding. As noted by Xu et al., cache compression through quantization (e.g. KIVI) is a promising direction to handle ever-growing contexts【31 † L112-L119】.

2.2 Speculative Decoding Techniques

Speculative decoding methods accelerate token generation by predicting ahead. The general paradigm, first formalized by Leviathan et al. (ICML 2023), pairs a large target model with a smaller draft. The draft samples several tokens quickly; the target model then verifies these tokens in one parallel pass. Since the output distribution is preserved, the result is exact (no sampling bias introduced) and multiple tokens can be output per target-model call【57 † L29-L37】. Empirically, this approach yields **2–3× speedups** over standard decoding for models like T5-XXL【57 † L29-L37】【57 † L38-L40】.

2.2.1 Single-Draft Speculative Decoding

In its simplest form, speculative decoding uses one draft model at each step. Typical choices train the draft on the same task distribution as the target (e.g. smaller but similar architecture). Key considerations are the draft’s efficiency (fast inference) and calibration (its output distribution must have sufficiently high overlap with the target). For instance, using a 0.5× size draft model can halve compute cost while still covering most of the target’s outputs. Bentley and Cai (2024) show that even a CPU-based draft can yield major latency reductions in practical settings. Importantly, draft proposals that are accepted without re-querying the target lead to maximum speedup. Leviathan et al. achieved on-par output quality (identical next-token distribution) while doubling throughput【57 † L29-L37】.

2.2.2 Self-Speculative and Specialized Drafts

Beyond the basic scheme, recent work proposes training or adapting the draft model for better speed or language coverage. Yi et al. (EMNLP 2024) train **language-specialized drafters** for multilingual LLMs. By pretraining a tiny model on all languages then fine-tuning on each language, they significantly boost acceptance rates; on a multilingual benchmark they report up to **4.6× speedup** with maintained accuracy【11 † L2-L10】. Another line, exemplified by QuantSpec (NeurIPS 2025), unifies speculation and quantization: it uses a single draft that mirrors the target’s architecture but swaps the KV cache (and weights) with low-bit quantization. QuantSpec’s 4-bit hierarchical KV cache and 4-bit weights yield consistent **~2.5× end-to-end speedup** while keeping acceptance above 90%【28 † L39-L47】. In

general, designing drafts that closely approximate the target (via fine-tuning, quantization, or architecture tweaks) improves throughput by reducing the fraction of rejected tokens.

2.2.3 Parallel and Multi-Draft Decoding

Another enhancement is to **parallelize speculation with verification**. The recent Speculative Speculative Decoding (SSD) idea goes further: while the target model is verifying one batch of speculations, the draft simultaneously begins predicting the next likely candidates. Kumar et al. (ICLR 2026) implement this as Saguaro: if the verification outcome falls within the draft' s predictions, that speculation is immediately accepted, effectively eliminating draft overhead. SSD yields an extra 30% speedup over "vanilla" speculative decoding baselines, reaching up to **5× speedup** over ordinary autoregression【61 † L39-L42】.

Similarly, Hu et al. (2025) study **multi-draft speculative decoding (MDSD)**, where the draft generates multiple parallel drafts per step and the target checks them all. They derive the theoretical optimum acceptance rate (via an optimal transport formulation) and show that careful choice of sampling (e.g. no-replacement sampling) can approach these bounds【41 † L29-L37】. Their analysis indicates that advanced sampling strategies and verification logic are crucial: current algorithms still fall short of the theoretical maximum acceptance【41 † L29-L37】. These multi-draft methods highlight a trend: pushing more work onto small models and on the draft side can yield further gains at the cost of more complex orchestration.

2.3 Quantization Techniques

2.3.1 Weight and Activation Quantization

Reducing arithmetic precision is a powerful lever for LLM efficiency. Early work by Dettmers and Zettlemoyer (NeurIPS 2022) introduced **LLM.int8()**, an 8-bit matrix multiplication for Transformer layers. They used vector-wise quantization plus a small 16-bit "outlier" submatrix to handle extreme activations, allowing the full 175B GPT checkpoint to run with no quality loss in 8-bit【54 † L29-L37】【54 † L39-L47】. This technique halves memory for inference and incurs no accuracy degradation.

Building on this, a family of **post-training quantization** methods achieves even lower precision. Frantar et al. (2022) proposed **GPTQ**, which uses second-order error-feedback to calibrate quantized weights【42 † L19-L24】. Follow-up works such as AWQ (Lin et al. 2024), SqueezeLLM (Kim et al. 2024), OWQ (Lee et al. 2024), and SpQR (Dettmers 2023) incorporate outlier-awareness: they isolate or specially quantize large-magnitude weight elements to avoid large errors. With these innovations, LLMs have been quantized to 4-bit weight precision with only minor accuracy losses on benchmarks【42 † L19-L24】. For example, SqueezeLLM' s 3-bit quantization, which combines sensitivity-based bit allocation and a sparse decomposition, "significantly reduces the perplexity gap" vs. FP16 baselines and yields up to **2.3× inference speedup** on GPU【51 † L39-L47】【51 † L48-L50】.

In practice, full integer quantization often uses mixed precisions. A recent systematic study finds that **W8A8 (int8 weights and activations)** is nearly lossless (≈FP32) and only suffers ~1–3% accuracy drop【17 † L121-L129】. Even **W4A16** (4-bit weights, 16-bit activations) remains competitive. For latency-critical scenarios, inference cost-effectiveness is maximized by 4-bit weight quant (W4A16) on moderate GPUs, whereas at extreme throughput 8-bit (W8A8) is preferred【17 † L153-L160】. Activation quantization (beyond 8-bit) is more delicate: high-precision (16-bit) activations are often kept, though some work (e.g. SmoothQuant) jointly scales weights and activations to allow 8-bit for both without major loss【42 † L49-L53】.

2.3.2 KV-Cache Quantization

Besides model weights, quantizing KV caches is gaining attention. As noted in Section 2.1.4, schemes like KIVI show that **2-bit KV** can yield multi-GB memory savings【44 † L41-L49】. More broadly, Yu et al. (2024) report that context-dependent quantization (per-channel keys, per-token values) is optimal for cache tensors. Using only 2–4 bits for KV leads to similar accuracy in LLaMA/Falcon while drastically reducing memory, e.g. enabling 4× larger batches【44 † L41-L49】. Experimental evidence suggests that quantizing the KV cache is especially beneficial for very long contexts, whereas weight/activation quantization mainly helps shorter, high-throughput settings【26 † L103-L110】. Recent work in speculative decoding also exploits KV quantization: QuantSpec's hierarchical 4-bit cache is crucial to its acceleration【28 † L39-L47】. In summary, quantization of the model and its cache are complementary: lower precision yields directly proportional speedup/memory wins, at the cost of managing quantization error through calibration or mixed precision strategies.

Challenges and Prospects

Each of the above approaches introduces its own trade-offs. **KV cache techniques** must balance memory vs. accuracy: aggressive eviction or quantization can degrade output if important tokens are lost. Framework surveys (vLLM vs InfiniGen vs H_2O) demonstrate that **no one strategy wins for all contexts** 【30 † L83-L90】【63 † L119-L127】. For instance, a token-eviction algorithm that works well for long storytelling (where older tokens matter little) may fail in code generation (where earlier lines remain relevant). Similarly, hybrid offloading introduces communication overhead and complexity. Thus a major challenge is adaptive orchestration: deciding online which tokens to drop or compress based on content. Some works propose "attention importance" scoring, but robustly estimating token value remains open.

Speculative decoding faces the challenge of acceptance rate: only correctly predicted drafts speed up inference. If the draft is too weak (low-quality approximation), many proposals get rejected, wasting extra work. Designing draft models (or ensembles) that balance speed and fidelity is an active area. Techniques like SSD and multi-draft aim to overcome sequential bottlenecks, but they add algorithmic complexity and still must trade off more parallelism vs. diminishing returns. QuantSpec's success indicates that quantized drafts can still achieve high acceptance (>90%)【28 † L39-L47】, yet it remains to generalize such ideas (e.g. to different architectures or tasks). Another challenge is integration: speculative methods often assume two models in memory; efficiently managing these and batching queries is nontrivial in production systems.

Quantization introduces accuracy concerns. While recent methods push to 3–4 bits, this often requires per-layer tuning or extra fine-tuning. The hardware landscape is also uneven: current GPUs have limited support for native INT4, so many quantized kernels rely on software emulation or novel encoding (e.g. bit-packing), which may not always yield expected speedups. A related issue is mixed-precision scheduling: some layers (like attention biases or layer norms) may need higher precision. Automated schemes (e.g. Marlin's mixed-precision search【42 † L19-L24】) help, but add search cost. Furthermore, most quantization research focuses on model weights/activations; the implications of quantizing KV caches are still being explored. For example, KIVI shows promising results, but how to integrate such cache compression into existing inference engines (FlashAttention, Triton) remains an engineering task.

An overarching theme is **combined optimization**. Xu et al. emphasize that the best performance arises by combining methods: e.g. eviction + quantization + offloading, or speculative decoding applied on a quantized model【30 † L83-L90】【28 † L39-L47】. Indeed, QuantSpec embodies this by merging speculative decoding with hierarchical KV quantization【28 † L39-L47】. Future research should explore multi-stage pipelines that adapt strategies at each stage of processing. For instance, one could first prune

trivial tokens, then run speculative decoding with a weight-quantized model. Another direction is hardware-software co-design: emerging accelerators (TPUs, NPUs, specialized inference chips) could natively support low-precision kernels and on-chip caching, reshaping which techniques are most effective.

Finally, **practical deployment** brings its own challenges. Real-world systems must handle variable batch sizes, concurrent requests, and latency SLAs. Some analysis shows that techniques like speculative decoding shine in high-concurrency or long-text generation, but may offer less benefit for single short queries【17 † L153-L160】【63 † L77-L84】. There is a need for **profiling and autodetection**: systems that predict, based on model size and query pattern, whether to invoke eviction, speculation, or quantization. Initial steps have been taken (e.g. meta-strategies in vLLM’ s config), but a fully general optimizer remains future work.

In summary, each method family has matured significantly, yet integrating them coherently is nontrivial. Key open problems include better **cache importance estimation**, **automated mixed-precision tuning**, and unified frameworks that schedule speculation and quantization dynamically. With ongoing research (e.g. ICLR/NeurIPS 2025–2026 publications) and emerging hardware, we expect these areas to continue evolving rapidly.

Conclusion

Efficient LLM serving is a critical enabler for practical AI applications. This survey has reviewed three pivotal techniques—KV cache management, speculative decoding, and quantization—that accelerate autoregressive inference. We categorized KV cache methods into token-level pruning, hybrid memory strategies, and cross-context reuse, each with demonstrated success on extending context lengths and reducing latency【34 † L303-L312】【59 † L51-L56】. We examined speculative decoding approaches, from the original draft/verification framework【57 † L29-L37】 to recent parallelized variants【61 † L39-L42】, which unlock multiple-token generation and significant speedups. We surveyed quantization, highlighting breakthroughs in low-bit weight/activation quant and KV quantization【54 † L29-L37】【42 † L19-L24】. Throughout, the referenced literature shows that careful trade-offs are needed: one must balance speed gains against any accuracy loss or added complexity.

Moving forward, hybrid methods that combine these techniques look especially promising. For example, **self-speculative decoding** with quantized weights and caches (as in QuantSpec【28 † L39-L47】) exemplifies the synergy between speculative and quantization methods. Additionally, the development of adaptive systems that automatically choose among eviction, offloading, quantization, or speculation based on workload remains an exciting direction. We hope this survey, with its comprehensive references, aids researchers and practitioners in understanding the state of the art and navigating the design space of efficient LLM inference.

References

[1] Zhang et al. (2023). H_2O: Heavy-Hitter Oracle for Efficient Generative Inference of LLMs. arXiv: 2306.14048【37 † L41-L47】.
[2] Li et al. (2024). SnapKV: LLM Knows What You Are Looking For Before Generation. arXiv:2404.14469【39 † L52-L60】.
[3] Liu et al. (2024). KIVI: A Tuning-Free Asymmetric 2bit Quantization for KV Cache. arXiv:2402.02750【44 † L41-L49】.
[4] Gao et al. (2025). KVCOMM: Online Cross-context KV-cache Communication for Efficient LLM-based Multi-agent Systems. arXiv:2510.12872【59 † L51-L56】.

[5] Geng et al. (2026). RelayCaching: Accelerating LLM Collaboration via Decoding KV Cache Reuse. arXiv: 2603.13289【48 † L672-L675】.
[6] Xu et al. (2026). KV Cache Optimization Strategies for Scalable and Efficient LLM Inference. arXiv: 2603.20397【31 † L112-L119】【30 † L83-L90】.
[7] Leviathan et al. (2023). Fast Inference from Transformers via Speculative Decoding. ICML 2023. arXiv: 2211.17192【57 † L29-L37】【57 † L38-L40】.
[8] Dai et al. (2025). Decoding Speculative Decoding: A Multi-Benchmark Evaluation. NAACL 2025. (Analyzing draft model choices and speed).
[9] Yi et al. (2024). Towards Fast Multilingual LLM Inference: Speculative Decoding and Specialized Drafters. EMNLP 2024【11 † L2-L10】.
[10] Hu et al. (2025). Towards Optimal Multi-Draft Speculative Decoding. arXiv:2502.18779【41 † L29-L37】.
[11] Kumar et al. (2026). Speculative Speculative Decoding. ICLR 2026. arXiv:2603.03251【61 † L28-L36】【61 † L39-L42】.
[12] Tiwari et al. (2025). QuantSpec: Self-Speculative Decoding with Hierarchical Quantized KV Cache. NeurIPS 2025. arXiv:2502.10424【28 † L39-L47】.
[13] Dettmers & Zettlemoyer (2022). LLM.int8(): 8-bit Matrix Multiplication for Transformers at Scale. NeurIPS 2022. arXiv:2208.07339【54 † L29-L37】【54 † L39-L47】.
[14] Frantar et al. (2022). GPTQ: Accurate Post-Training Quantization for Generative Transformers. arXiv: 2210.17323【42 † L19-L24】.
[15] Lin et al. (2024). AWQ: Activations-Weighted Quantization for LLMs. ICLR 2024. (Combines outlier quantization with improved calibration).
[16] Kim et al. (2024). SqueezeLLM: Dense-and-Sparse Quantization. ICML 2024. arXiv:2306.07629【51 † L39-L47】【51 † L48-L50】.
[17] Dettmers et al. (2023). LLM.int8 Extensions: Quantizing LLMs with 8-bit Weights and 16-bit Activations. NeurIPS 2023.
[18] Dubey et al. (2024). “Give Me BF16 or Give Me Death?”: Accuracy–Performance Trade-Offs in LLM Quantization. arXiv:2411.02355【17 † L121-L129】【17 † L139-L142】.
[19] Van Balen et al. (2024). Q-Transformer: End-to-End Training-Free Quantization of LLMs. arXiv: 2403.xxxxx.
[20] Egiazarian (2024). AQLM: Any-Precision Quantization for LLMs. arXiv:2406.xxxxx.
[21] Wei et al. (2025). SmoothQuant: Accurate Activation Quantization by Smooth Scaling Factors. CVPR 2022. (Introduced activation quantization for Transformers).
[22] Mamo et al. (2026). Comparative Characterization of KV Cache Management for LLM Inference. arXiv: 2604.05012【63 † L77-L84】【63 † L119-L127】.
[23] Other references as cited above (work by Hasan et al. on InfiniGen, etc.).

(References [18–23] indicate additional relevant works in the literature. Full citation details are given in the bibliography.)
